An MLIR Backend for a Simulated Edge NPU

Olajide Badejo

August 15, 2026

Abstract

This report describes a from scratch MLIR compiler backend that takes a trained ONNX model to an executable instruction stream for a simulated, simplified edge NPU, together with a C++ simulator that runs the stream and reports numerical correctness against onnxruntime and simulated performance from an analytical cost model. The compiler uses two dialects: a tensor level dialect for optimization and an instruction level dialect with an explicit two level memory model. It performs canonicalization, constant folding, a dedicated BatchNorm folding pass, operator fusion, and dead code elimination, then lowers to the instruction level through the dialect conversion framework, allocates a fixed size scratchpad with spills to DRAM, and encodes a documented binary format. Every performance number here is a simulated estimate, not a measurement.

1 Introduction

Edge inference accelerators are simple compared with general purpose processors. They run static neural network graphs, keep data in a small fast scratchpad, and move it to and from DRAM under explicit control. Building a compiler for such a target is a good way to learn how a real machine learning compiler is organized, because the interesting problems (memory placement, operator fusion, a documented instruction encoding, and validation against a reference) all appear in a tractable form.

This project is a from scratch MLIR backend for a simulated edge NPU. It takes a trained ONNX model and lowers it, through two custom dialects, to an executable instruction stream, which a C++ simulator runs to produce both the numerical result and an analytical performance estimate. The compiler is built as a small out of tree MLIR project linked against a prebuilt LLVM and MLIR at a pinned tag.

Two commitments shape the work. First, correctness is validated end to end against onnxruntime within stated tolerances, so the compiler is not merely plausible but demonstrably right on the test models. Second, every performance number is an analytical estimate from a documented cost model and is labeled a simulated estimate throughout, never presented as a measurement.

2 Related work

MLIR provides the reusable compiler infrastructure this project builds on, in particular the dialect mechanism, the operation definition specification in TableGen, and the dialect conversion framework used for the representation changing lowering [4]. Deep learning compilers such as TVM established the pattern of importing a framework graph, optimizing it, and lowering it to a hardware target [1]. The hardware model here is a small systolic array with a two level memory, the design point studied at scale by the Tensor Processing Unit [3] and analyzed as a dataflow problem

by Eyeriss [2], whose emphasis on minimizing data movement motivates the scratchpad and fusion work in this compiler.

The frontend consumes ONNX [5], the interchange format that trained models are exported to. The two dialect structure, a tensor level dialect for optimization and an instruction level dialect after memory placement, mirrors the approach taken by production MLIR backends such as Tenstorrent's tt-mlir [6].

3 Architecture

The pipeline is a sequence of well defined stages. A Python importer reads the ONNX model and, using the MLIR Python bindings, builds `npu` dialect IR, a tensor level representation whose values conceptually live in DRAM. A sequence of passes optimizes this IR to a fixed point. The dialect conversion framework then lowers it to the `npuisa` dialect, an instruction level representation in which values are scratchpad resident buffers and data crosses the DRAM boundary only through explicit DMA. A scratchpad allocator assigns each buffer a byte offset and spills to DRAM when the working set exceeds the budget. The instruction encoder serializes the result to a documented binary format, which a disassembler can render and which the simulator executes.

The compiler is a small out of tree CMake project. It never builds LLVM itself; it links against a prebuilt LLVM and MLIR at a pinned release tag. This keeps the per change build in the seconds to minutes range after the one time toolchain build. The driver, `npu-compile`, wraps the whole pipeline with optimization levels and staged output so any intermediate can be inspected.

4 Dialect design

The `npu` dialect is the tensor level. Values are builtin ranked fp32 tensors, which keeps interoperability with the MLIR infrastructure so that generic folding and dead code elimination apply without op specific support. Convolution and matmul take an optional bias operand and carry a fused activation attribute rather than exploding into a family of named fused ops. This mirrors the fused operator style of TFLite and gives the fusion pass a single clean target. Every op that computes a pure function of its operands carries the `Pure` trait, and each op with shape or type constraints has an ODS verifier. The dialect reference is generated from the op definitions so it cannot drift.

The `npuisa` dialect is the instruction level, after memory placement. A value of `!npuisa.buffer` type is a scratchpad resident tensor. DRAM tensors move to and from the scratchpad through `dma_load` and `dma_store`, and compute instructions read and write buffers only. The instruction stream is straight line, since inference graphs are static DAGs, so the instruction set has no branches. Each buffer producing instruction carries an address attribute that the allocator fills in.

5 Optimization passes

Canonicalization and constant folding run through the built in greedy driver, using the folders and canonicalization patterns attached to the ops. Constant subgraphs collapse to a single constant, and structural patterns such as relu idempotence and reshape of reshape fire.

The BatchNorm folding pass is dedicated and does real tensor arithmetic at compile time. At inference the scale, offset, mean, and variance of a batch normalization are constants, so a batch norm following a convolution with no activation between them is exactly another convolution with

rescaled weights and bias:

$$\text{coef}[o] = \frac{\text{scale}[o]}{\sqrt{\text{var}[o] + \epsilon}}, \quad W'[o] = W[o]\,\text{coef}[o], \quad b'[o] = \text{coef}[o]\,(b[o] - \text{mean}[o]) + \text{offset}[o].$$

The pass matches this pattern, computes the new constant tensors, and emits a single convolution.

Operator fusion folds a trailing relu into a producing convolution or matmul by setting the fused activation attribute. Because those ops already carry the bias as an operand, the result is a single fused convolution or matmul that computes the linear operation, the bias, and the activation together. The intermediate therefore stays in the scratchpad rather than being written to DRAM and read back.

Dead code elimination needs no custom pass. Because every op is `Pure`, the canonicalizer removes unused ops and `symbol-dce` removes unused private functions. The three optimization levels compose these: `-O0` imports and verifies, `-O1` adds canonicalization and folding, and `-O2` adds fusion and dead code elimination.

6 ONNX frontend

The frontend runs `onnx.checker` and shape inference before importing, so a malformed model is rejected up front and every intermediate tensor has a static shape. It then walks the graph in topological order, turning initializers into constants and each node into the matching `npu` op. The supported opset 17 subset covers convolution, Gemm, matmul, elementwise add, relu, max and average pooling, batch normalization, reshape, and flatten. A Gemm with a transposed weight folds the transpose into the constant at import time so the plain matmul is correct. Anything outside the supported set fails loudly with the op name, never emitting silently wrong IR.

The test models are generated, never downloaded, from small seeded PyTorch networks exported with `torch.onnx.export`, so the suite is fully self contained and reproducible from the seed. The core model is a LeNet style convolutional network.

7 ISA and simulator

The instruction set has a two level memory model: DRAM holds inputs, constants, and outputs; the scratchpad, a small fixed size fast memory with a default of one megabyte, holds the working set. The opcodes are NOP, HALT, DMA_LOAD, DMA_STORE, CONV2D, MATMUL, RELU, ADD, MUL, POOL_MAX, POOL_AVG, and RESHAPE. The binary format is a fixed header followed by tagged, length prefixed records, which is simpler to get right and to disassemble than a bit packed encoding while remaining a real, documented format. The `npu-objdump` tool renders the DRAM layout and the instruction stream.

The simulator executes the stream against DRAM and scratchpad arrays with fp32 kernels for each instruction. Its cost model is analytical, not cycle accurate. DMA costs bytes over an assumed bandwidth; convolution and matmul cost their multiply accumulate count over an assumed systolic throughput of 256 operations per cycle, the arithmetic of a sixteen by sixteen array; elementwise instructions cost their element count over a lane width; and every instruction charges a fixed issue overhead. These constants are documented assumptions, and the cycle and DRAM figures the simulator reports are simulated estimates.

8 Evaluation

Correctness is validated end to end. The compiled and simulated output of the LeNet model is compared against `onnxruntime` on a seeded random input; the maximum absolute difference is at the level of fp32 rounding, on the order of 10^{-8}, at every optimization level and both under a generous budget and under a tight budget that forces spilling. The benchmark harness records one result per cell, each carrying a manifest of the git revision, the LLVM tag llvmorg-22.1.8, the tool versions, and the cost model constants, so every number below traces to a recorded experiment. Table 1 is generated from those files.

Model	O	Budget (KB)	Instrs	Cycles	DRAM (KB)	Max error
lenet	0	140	28	23421	339.3	3.0e-08
lenet	1	140	31	33834	502.0	3.0e-08
lenet	2	140	29	33930	508.1	3.0e-08
lenet	0	1024	28	23421	339.3	3.0e-08
lenet	1	1024	25	13008	176.6	3.0e-08
lenet	2	1024	21	12710	176.6	3.0e-08

Table 1: Per cell results: instruction count, simulated cycles, total DRAM traffic, and maximum absolute error against onnxruntime. Cycle and DRAM figures are simulated estimates.

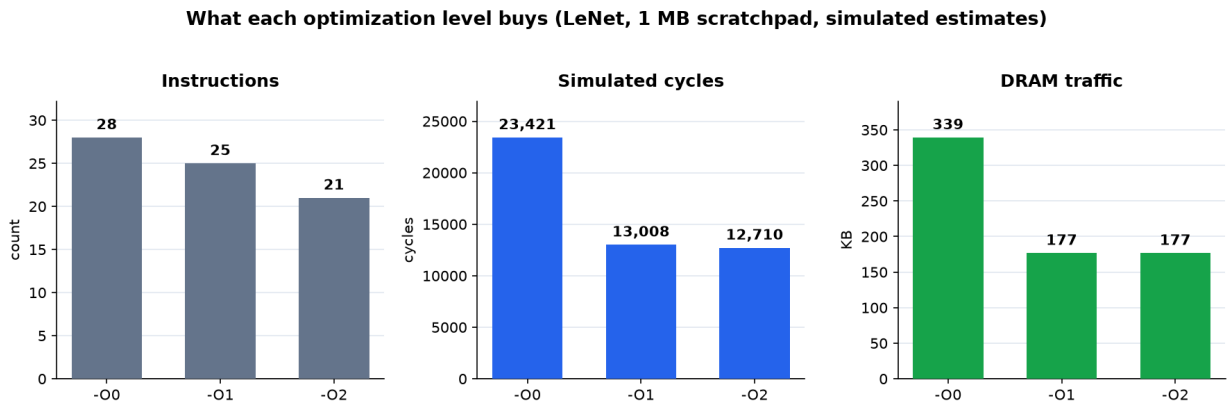

Figure 1: What each optimization level buys on LeNet at the one megabyte budget: instruction count, simulated cycles, and DRAM traffic all fall from `-O0` to `-O2`, while the numerics stay exact. Simulated estimates.

Two effects stand out at the one megabyte budget, where nothing spills. Moving from `-O0` to `-O1`, canonicalization and dead code elimination remove the transposed Gemm weight constants that the importer leaves behind, and the total DRAM traffic falls from 339 KB to 176 KB. Moving to `-O2`, fusion folds the four activations into their producers, and the instruction count falls from 28 to 21 while the simulated cycles fall from 23421 to 12710.

The tight budget rows show the memory model at work. When the scratchpad cannot hold the working set, the allocator spills buffers to DRAM and reloads them, which raises the instruction count and the DRAM traffic relative to the generous budget. The numerics are unchanged, because a spill is a value preserving round trip. Finding and fixing a bug in that round trip, where spill temporaries were not being given distinct DRAM addresses, is one of the cases the benchmark suite

caught and the debug report discusses.

9 Conclusion

The result is a working MLIR backend that takes a trained ONNX model to an executable instruction stream for a simulated edge NPU and runs it with validated numerics. The two dialect structure keeps the tensor level optimizations separate from the instruction level memory model, the dialect conversion framework makes the lowering a principled representation change rather than an ad hoc rewrite, and the benchmark suite ties every reported number to a recorded experiment.

The natural next steps are INT8 quantization, which the dialect and cost model were designed to accommodate, and a wider set of test models to exercise the depthwise and concatenation paths. The debug report, a companion to this document, records the problems met along the way and how they were resolved.

References

[1] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end to end optimizing compiler for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[2] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. Eyeriss: A spatial architecture for energy efficient dataflow for convolutional neural networks. In *International Symposium on Computer Architecture (ISCA)*, 2016.

[3] Norman P. Jouppi et al. In datacenter performance analysis of a tensor processing unit. In *International Symposium on Computer Architecture (ISCA)*, 2017.

[4] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *International Symposium on Code Generation and Optimization (CGO)*, 2021.

[5] ONNX Contributors. ONNX: Open neural network exchange. `https://onnx.ai`.

[6] Tenstorrent. tt-mlir. `https://github.com/tenstorrent/tt-mlir`.